

Fredrik Hammarberg

Optimizing Weak Reference Processing in the JVM Z Garbage Collector

*A Performance Analysis and Optimization of the ZGC
Reference Processing Pipeline*



UPPSALA
UNIVERSITET

Abstract
PLACEHOLDER

Acknowledgments

I would like to express my sincere gratitude to my supervisor for their guidance and support throughout this thesis work. It is through their help and encouragement that I was able to complete this thesis.

A special thanks to the Department of Information Technology at Uppsala University for providing the resources necessary to conduct the performance evaluations and experiments described in this thesis.

I would also like to thank my colleagues at Oracle for their insights and feedback on my work throughout the development process.

Finally, I would like to thank my family and friends for their patience and encouragement throughout my studies.

Uppsala, Sweden, June 2026

Fredrik Hammarberg

Contents

Acknowledgments	iii
1 Introduction	11
1.1 Motivation	11
1.2 Problem Statement	12
1.3 Research Questions	12
1.4 Stretch Goals	12
1.5 Scope and Delimitations	13
2 Background	14
2.1 Garbage Collection in the JVM	14
2.1.1 Generational Collection	14
2.2 Java Reference Types	15
2.2.1 The Reference Object Lifecycle	15
2.2.2 WeakReference and ReferenceQueue	16
2.3 The Z Garbage Collector	17
2.4 Reference Processing Pipeline	18
2.4.1 Discovery	18
2.4.2 Processing	19
2.4.3 Enqueue	19
2.5 Memory Hierarchy and Pointer-Chasing	20
2.6 Data Structures in Reference Processing	21
2.6.1 Intrusive Linked List	21
2.6.2 Dynamic Array	21
2.7 ZGC Reference Processing Details	22
2.7.1 Per-Worker Discovered Lists	22
2.7.2 Generational Reference Handling	23
2.7.3 Concurrent Processing and Barriers	23
2.8 Related Work	23
2.9 Summary	24
3 Method	26
3.1 Optimisation Approaches	26
3.1.1 ZGC Modifications	27
3.1.2 G1GC Modifications	27
3.2 Optimization Strategies	27
3.2.1 Null-Queue Skip Optimization	27

3.2.2	Data Structure Optimizations	27
3.2.3	Synchronization Optimizations	27
3.3	Benchmarking Methodology	27
3.3.1	Synthetic Benchmarks	27
3.3.2	Real-World Benchmarks	27
3.4	Experimental Setup	27
3.4.1	Hardware Configuration	27
3.4.2	Software Configuration	27
3.4.3	Measurement Methodology	27
4	Results	28
4.1	Synthetic Benchmark Results	28
4.1.1	Baseline Performance	28
4.1.2	Null-Queue Optimization Impact	28
4.1.3	Data Structure Impact	28
4.2	Caffeine Benchmark Results	28
4.2.1	Cache Operations Performance	28
4.2.2	GC Pause Time Analysis	28
4.3	Statistical Analysis	28
4.4	Summary of Findings	28
5	Discussion	29
5.1	Interpretation of Results	29
5.1.1	Performance Improvements	29
5.1.2	Trade-offs	29
5.2	Implications for JVM Development	29
5.2.1	Applicability to Other Collectors	29
5.2.2	Integration Challenges	29
5.3	Limitations	29
5.3.1	Benchmark Coverage	29
5.3.2	Correctness Considerations	29
5.3.3	Generational Complexity	29
5.4	Comparison with Related Work	29
5.5	Summary	29
6	Conclusion	30
6.1	Summary of Contributions	30
6.2	Answers to Research Questions	30
6.3	Future Work	30
6.3.1	Further Optimizations	30
6.3.2	Broader Evaluation	30
6.3.3	Integration with Mainline OpenJDK	30
6.4	Closing Remarks	30
	References	31

List of Tables

List of Figures

Figure 2.1:WeakReference constructors 16

1. Introduction

Java is one of the most widely deployed programming languages in enterprise and server-side computing, where applications routinely manage heaps of tens to hundreds of gigabytes. In these environments, the performance of the Java Virtual Machine’s (JVM) garbage collector (GC) directly affects application latency and throughput. The Z Garbage Collector (ZGC) addresses this by performing nearly all garbage collection work concurrently with the application, achieving sub-millisecond pause times regardless of heap size.

However, one aspect of ZGC that has received comparatively little optimization attention is its processing of *weak references*. Java’s `WeakReference` class allows applications to hold references that do not prevent the referent from being collected, and it is widely used in caching libraries, canonical mappings, and listener registrations. When the garbage collector determines that a weakly referenced object is no longer strongly reachable, it must clear the reference and if the reference was registered with a `ReferenceQueue`, enqueue it for the application to process.

The current ZGC reference processing pipeline treats all discovered weak references uniformly: they are linked together in an intrusive linked list, processed, and then handed to a Java-level daemon thread (the `ReferenceHandler`) for enqueueing. This design introduces overhead that is disproportionate for a specific subset of weak references: those that are not registered with a `ReferenceQueue`.

This thesis investigates whether targeted modifications to ZGC’s reference processing pipeline can reduce this overhead. Three orthogonal optimization approaches are explored: skipping the enqueue path for references without queues, replacing the intrusive linked list with a cache-friendly dynamic array for such references, and reducing per-reference instruction cost through sequential code optimizations.

1.1 Motivation

Weak references occupy a unique position in JVM garbage collection. Unlike regular references, which are handled implicitly by the mark phase, weak references require explicit processing by the collector: each discovered weak reference must be individually evaluated, its referent cleared if unreachable, and the reference potentially enqueued. This per-reference processing cost scales

linearly with the number of weak references, making it a bottleneck in applications that allocate millions of them.

The problem is amplified by a mismatch between the API design and its typical usage. Java's `WeakReference` API provides an optional `ReferenceQueue` parameter, and the garbage collector's pipeline is designed around the assumption that cleared references will always be enqueued. The result is a pipeline that performs linked-list threading, cross-thread handoff, and iteration work for references that will never be enqueued.

ZGC's concurrent architecture makes this inefficiency particularly relevant. Because ZGC processes references concurrently with the application, any reduction in per-reference overhead translates directly into less time spent in the reference processing phase, freeing GC worker threads for other tasks.

1.2 Problem Statement

The current ZGC reference processing pipeline does not distinguish between weak references with and without reference queues during the discovery and enqueue phases. All cleared references are threaded onto a pending list, transferred to the `ReferenceHandler` thread, and iterated regardless of whether they will actually be enqueued. Additionally, the intrusive linked-list data structure used for discovered references exhibits poor cache locality and overloads the `discovered` field for both GC-internal linking and pending-list linking.

This thesis seeks to determine whether these inefficiencies can be addressed through modifications to ZGC's reference processor, and to quantify the resulting performance impact.

1.3 Research Questions

The main research questions that this thesis aims to address are:

1. What design changes and optimizations can be made to the weak reference processing in ZGC to improve its performance?
2. How do these design changes and optimizations impact the overall performance and memory footprint of ZGC?

1.4 Stretch Goals

In addition to the primary research questions, the following stretch goals are defined to explore the broader implications of this work:

1. Does the introduction of a weak field annotation in Java provide a more efficient way to handle weak references compared to the current

`WeakReference` object-based implementation? A weak field would allow individual fields of an object to have weak-reference semantics without allocating a separate `WeakReference` object, potentially eliminating both the memory overhead and the reference processing cost entirely.

2. Can the optimizations developed for ZGC be applied to other garbage collectors in the JVM, such as the G1 garbage collector, and if so, what performance improvements can be observed? The shared reference processor used by G1 has a different threading model (using CAS-based discovery), so the applicability and performance characteristics of the optimizations may differ.

1.5 Scope and Delimitations

This thesis focuses on the reference processing pipeline within ZGC, specifically targeting weak references. While the optimization approaches are described in general terms and the stretch goals consider applicability to G1 and alternative API designs, the primary implementation and evaluation are conducted within the ZGC codebase of OpenJDK.

The correctness of the optimizations is verified through the standard OpenJDK test suite and manual inspection; formal verification is outside the scope of this work. Performance measurements are conducted on a single hardware platform under controlled conditions so therefore may not generalize to all environments.

2. Background

The three optimization approaches laid out in section 3.1 rely on several key concepts and design decisions in the Java Virtual Machine's (JVM) garbage collection architecture, the Z Garbage Collector (ZGC), and the Java reference processing pipeline. This chapter provides the necessary background on these topics to understand the motivation and implementation of the optimizations.

2.1 Garbage Collection in the JVM

The Java Virtual Machine (JVM) provides automatic memory management through garbage collection (GC). Rather than requiring the programmer to explicitly allocate and free memory as in languages such as C or C++, the JVM tracks which objects are reachable from the application's root set comprising thread stacks, static fields, and JNI references and reclaims the memory of objects that are no longer reachable.

The JVM ships with several garbage collector implementations, each designed with different trade-offs between throughput, latency, and memory footprint:

- **Serial GC** A single-threaded, stop-the-world collector suitable for small heaps and client-style applications.
- **Parallel GC** A multi-threaded throughput-oriented collector that parallelizes both young and old generation collection.
- **G1 GC (Garbage-First)** A region-based collector that balances throughput and latency by incrementally collecting regions with the most garbage. G1 divides the heap into equal-sized regions and uses concurrent marking alongside evacuation pauses.
- **ZGC (Z Garbage Collector)** A scalable, low-latency collector designed for heaps ranging from megabytes to terabytes. ZGC performs nearly all of its work concurrently with the application, resulting in sub-millisecond pause times regardless of heap size.

This thesis focuses on ZGC, as its concurrent design and generational architecture introduce specific challenges for reference processing that can benefit from targeted optimization.

2.1.1 Generational Collection

Most modern JVM garbage collectors employ *generational collection*, based on the *weak generational hypothesis*: most objects die young. The heap is divided into at least two generations:

- The **young generation**, where newly allocated objects reside. Young generation collections are frequent but fast, since most young objects are short-lived.
- The **old generation**, where objects that have survived multiple young collections are promoted. Old generation collections are less frequent but more expensive.

ZGC adopted generational collection starting with JDK 21. In generational ZGC, the young and old generations can be collected independently and concurrently. This is relevant to reference processing because references in different generations are handled during different collection cycles. Cross-generational relationships, where a reference object is in one generation and its referent is in another, require special treatment (section 2.7.2).

2.2 Java Reference Types

In addition to ordinary (strong) references, Java provides a hierarchy of *reference objects* in the `java.lang.ref` package that allow applications to interact with the garbage collector's reachability decisions. These reference types, listed from strongest to weakest, are:

1. **SoftReference** The referent is cleared at the discretion of the GC, typically when memory is low. Commonly used for memory-sensitive caches.
2. **WeakReference** The referent is cleared as soon as it becomes weakly reachable (i.e., no strong or soft references exist to it). Widely used in caching frameworks, canonical mappings, and listener registrations.
3. **FinalReference** An internal JVM type (not part of the public API) used to implement the `finalize()` mechanism. Objects with finalizers are wrapped in a `FinalReference` so the GC can schedule finalization before reclaiming the object.
4. **PhantomReference** The referent is never accessible through the reference; `get()` always returns `null`. Used for post-mortem cleanup actions, often as a replacement for finalizers.

All four types extend the abstract class `Reference<T>` (in the `java.lang.ref` package), which defines the key fields and the life-cycle managed by the garbage collector.

2.2.1 The Reference Object Lifecycle

Each `Reference` object maintains several fields that track its state:

- `referent` The referenced object. The GC may clear this field (set it to `null`) when the referent becomes eligible for collection according to the reference's strength.

- `queue` The `ReferenceQueue` this reference is registered with. If the reference was not created with a queue, this field points to the sentinel object `ReferenceQueue.NULL_QUEUE`.
- `next` Used as a link pointer when the reference is enqueued in a `ReferenceQueue`.
- `discovered` A field with dual purpose: during GC marking, it serves as a link in the garbage collector's internal *discovered list*; after processing, it is reused as a link in the VM's *pending list* that feeds the `ReferenceHandler` thread.

A reference object transitions through the following states during its lifetime:

1. **Active** The reference has been created and the referent is still reachable (or not yet processed). The `discovered` field may be used by the GC.
2. **Pending** The GC has determined the referent is eligible for clearing and has added the reference to the VM's pending list. The `ReferenceHandler` thread will process it.
3. **Enqueued** The `ReferenceHandler` thread has moved the reference into its associated `ReferenceQueue`, where the application can retrieve it via `poll()` or `remove()`.
4. **Inactive** The reference has been dequeued by the application, or was created without a queue and has been cleared. It is no longer tracked by the GC or queuing infrastructure.

An important observation is that references created *without* a `ReferenceQueue` can transition directly from Active to Inactive, bypassing the Pending and Enqueued states entirely. However, as discussed in section 2.4, the default implementation does not always take advantage of this shortcut.

2.2.2 WeakReference and ReferenceQueue

A `WeakReference` is the most commonly used non-strong reference type in Java applications. It has two constructors:

```

1 // Without a queue - queue field set to NULL_QUEUE
2 public WeakReference(T referent) {
3     super(referent);
4 }
5
6 // With a queue
7 public WeakReference(T referent, ReferenceQueue<? super
8     T> q) {
9     super(referent, q);
10 }

```

Figure 2.1. `WeakReference` constructors

When a `WeakReference` is created with a `ReferenceQueue`, the application can be notified when the referent is collected: the GC clears the referent and eventually enqueues the reference, allowing the application to perform cleanup. When created *without* a queue, the reference serves only as a conditional pointer the application can call `get()` to check whether the referent is still alive, but receives no notification upon clearing.

The `ReferenceQueue` is implemented as a synchronized singly-linked list using the `next` field of the `Reference` objects. It provides `poll()` for non-blocking retrieval and `remove()` for blocking retrieval with an optional timeout.

A key insight for this thesis is that **many real-world uses of `WeakReference` do not use a `ReferenceQueue`**. Caching frameworks (including the widely used Caffeine library [1]), `WeakHashMap`, and similar data structures typically create weak references without queues, relying instead on periodic polling or lazy cleanup. This means the garbage collector expends effort preparing these references for enqueueing (adding them to the pending list) even though the `ReferenceHandler` thread will immediately skip them.

2.3 The Z Garbage Collector

ZGC is a concurrent, region-based, compacting garbage collector included in the JDK since JDK 11 (experimental) and production-ready since JDK 15. Starting with JDK 21, ZGC supports generational collection, which is now the default mode. ZGC’s primary design goals are:

- Sub-millisecond pause times that do not increase with heap size.
- Support for heap sizes from 8 MB to 16 TB.
- Concurrent marking, relocation, and reference processing.

ZGC achieves these goals through several key techniques:

Colored Pointers.

ZGC embeds metadata in the unused bits of 64-bit object pointers (or in a dedicated area when using multi-mapped memory). These *color bits* encode information about the pointer’s current state relative to the GC’s phase for example, whether the pointer has been remapped after relocation, or whether it points to a young or old generation object. This allows the GC to reason about pointer states without loading the object header.

Load Barriers.

ZGC uses a load barrier injected by the JIT compiler at every reference load. The barrier checks the color bits of the loaded pointer and, if necessary, applies a fixup (e.g., remapping after relocation or marking for concurrent marking). This is the mechanism that allows ZGC to perform most of its work concurrently with the application.

Concurrent Phases.

A ZGC collection cycle interleaves short stop-the-world pauses (for root scanning and synchronization) with concurrent phases (for marking, reference processing, and relocation). The concurrent phases run alongside application threads, using load barriers to maintain consistency.

2.4 Reference Processing Pipeline

Reference processing is the mechanism by which the garbage collector handles `Reference` objects discovered during marking. The pipeline has three stages: *discovery*, *processing*, and *enqueueing*. In ZGC, the first two stages run concurrently with the application during the marking phase, while the enqueue stage involves handing off references to the Java-level `ReferenceHandler` thread.

2.4.1 Discovery

During the concurrent marking phase, when the garbage collector encounters a `Reference` object (i.e., an instance of `SoftReference`, `WeakReference`, `FinalReference`, or `PhantomReference`), it evaluates whether the reference should be *discovered* that is, added to a list for later processing.

The discovery decision depends on several factors:

1. **Is the reference active?** Only active references are eligible. For non-`FinalReference` types, an inactive reference has a null referent. For `FinalReference`, inactivity is indicated by a non-null `next` field.
2. **Is the referent still strongly reachable?** If the referent is already marked as strongly live, the reference does not need processing. The garbage collector checks this via its liveness information.
3. **Soft reference policy.** For `SoftReference` objects, the collector applies a policy (based on available memory and the reference's last access time) to decide whether to keep the referent alive or clear it.
4. **Generational considerations.** In generational ZGC, only references residing in the old generation are discovered during an old collection. References in the young generation are handled during young collections. (See section 2.7.2 for the implications.)

Once a reference is selected for discovery, it is added to a *discovered list*. In the standard OpenJDK implementation (used by G1, Serial, and Parallel GCs), discovered references are linked together in a per-thread singly-linked list using the `discovered` field of each `Reference` object as the link pointer. In ZGC, each GC worker thread maintains its own discovered list head, avoiding the need for atomic compare-and-swap (CAS) operations during discovery. This is in contrast to the shared reference processor (used by G1 and others),

which uses CAS on the `discovered` field when multiple threads may discover references concurrently.

2.4.2 Processing

After marking completes, the discovered lists are processed. For each discovered reference, the collector determines the final fate of the referent:

1. If the referent has become `null` (cleared by the application concurrently), the reference is dropped from the list.
2. If the referent is now strongly reachable (e.g., it was resurrected by another thread), the reference is dropped and the referent is kept alive.
3. If the referent is still unreachable according to the reference's strength, the referent is **cleared** (set to `null`), and the reference is prepared for enqueueing.

The processing order follows the strength hierarchy: soft references are processed first, then weak, then final, and finally phantom references. Processing soft references before weak references ensures that soft references are cleared only under memory pressure, while weak references are always cleared when the referent is not strongly reachable.

In ZGC, the “clearing” of a referent involves applying a *clean barrier* that writes a null value to the referent field. Different barrier variants are used depending on the reference type: a weak-OOP variant for soft and weak references, and a phantom-OOP variant for phantom references. These barriers ensure that the pointer coloring metadata is correctly maintained.

2.4.3 Enqueue

After a reference's referent has been cleared, the reference must be made available to the application. This happens through the *pending list* mechanism:

1. The GC builds an internal pending list by linking cleared references together via their `discovered` field.
2. Under the heap lock, the GC atomically prepends its internal pending list onto the global VM pending list.
3. The high-priority `ReferenceHandler` daemon thread is notified.
4. The `ReferenceHandler` thread wakes up, atomically retrieves the pending list, and iterates over it. For each reference, it checks whether `queue != NULL_QUEUE`. If the reference has a real queue, it calls `queue.enqueue(this)`; otherwise it simply advances to the next reference.

This design reveals an inefficiency: references created without a `ReferenceQueue` are still linked into the pending list, transferred to the `ReferenceHandler` thread, and iterated over only to be skipped. For workloads where the majority of weak references lack queues, this constitutes unnecessary

bookkeeping. This inefficiency is one of the key targets of the optimizations explored in this thesis.

2.5 Memory Hierarchy and Pointer-Chasing

Modern processors are equipped with deep memory hierarchies consisting of multiple levels of cache (typically L1, L2, and L3) between the CPU and main memory. Accessing data in the L1 cache takes on the order of a few nanoseconds, while a cache miss that must be serviced from main memory incurs a latency of roughly 50–100 nanoseconds, two orders of magnitude slower. The performance of any data-intensive algorithm therefore depends heavily on how well its memory access patterns interact with the caching hardware.

Two properties of memory access patterns determine cache efficiency:

- **Spatial locality:** Accessing data at nearby memory addresses. When a cache line (typically 64 bytes) is loaded, all data within it becomes available at L1 speed. Sequential traversal of a contiguous array exhibits strong spatial locality, since successive elements occupy adjacent or nearby cache lines.
- **Temporal locality:** Accessing the same data repeatedly within a short time window. Recently accessed data remains in cache and can be re-read without penalty.

A critical distinction arises between *contiguous* data structures (arrays) and *pointer-linked* data structures (linked lists) when traversal involves cache misses. In a contiguous array, all element addresses are computable from the base address and index; the processor does not need to read any element in order to determine the address of the next one. Modern out-of-order processors exploit this property aggressively: the memory subsystem can issue multiple cache-miss loads simultaneously, since the addresses of future elements are known in advance. Hardware prefetchers further accelerate sequential access by detecting the stride pattern and fetching upcoming cache lines before they are explicitly requested.

In contrast, a linked list exhibits a *pointer-chasing* access pattern: the address of the next node is stored inside the current node, so the processor cannot determine where the next load will go until the current load completes. Even on a wide out-of-order core capable of tracking dozens of in-flight memory operations, a linked-list traversal serializes at the rate of one cache miss per node. If each node resides on a separate cache line (as is typical for heap-allocated Java objects), the traversal stalls for the full memory latency on every step.

This difference has substantial practical consequences. When iterating over n elements that all miss in cache, the array-based traversal completes in approximately one memory-latency period (since all loads can overlap), while the linked-list traversal takes n times the memory latency. For the reference processing use case, where millions of `Reference` objects may be scattered

across a multi-gigabyte heap, this serialization can dominate the processing time.

2.6 Data Structures in Reference Processing

The choice of data structure for storing discovered references has significant implications for performance, particularly in light of the memory hierarchy considerations described in section 2.5. This section describes the structure used in the standard implementation and discusses the alternative considered in this work.

2.6.1 Intrusive Linked List

The standard implementation stores discovered references in an *intrusive singly-linked list*. Each `Reference` object's `discovered` field serves as the `next` pointer, forming a chain rooted at a per-thread list head. This design has several properties:

- **Zero allocation overhead:** No auxiliary nodes are allocated; the list is embedded within the `Reference` objects themselves.
- **O(1) prepend:** Adding a reference to the list is a single pointer write (or a CAS in the shared processor's multi-threaded mode).
- **Dual-purpose field:** After processing, the same `discovered` field is reused to link references in the pending list destined for the `ReferenceHandler` thread.

However, the linked list has drawbacks relevant to modern hardware:

- **Poor cache locality:** `Reference` objects are scattered across the heap. Traversing the discovered list is a pointer-chasing operation (section 2.5): each load depends on the result of the previous load, serializing cache misses and preventing the processor from overlapping memory accesses.
- **Field overloading:** The dual use of the `discovered` field means that during processing, building the pending list requires careful coordination a reference cannot simultaneously be part of the discovered list and the pending list.
- **Unnecessary pending-list work:** All cleared references are threaded onto the pending list, regardless of whether they have a queue. The `ReferenceHandler` thread must then iterate the entire list to pick out the ones that actually need enqueueing.

2.6.2 Dynamic Array

An alternative to the intrusive linked list is a *dynamic array* that stores meta-data about discovered references contiguously in memory. Rather than linking references via the `discovered` field, each entry in the array records:

- A pointer to the referent field of the `Reference` object.
- A pointer to the discovered field.
- The address of the referent object.
- A snapshot of the referent pointer value at discovery time.

This approach has complementary trade-offs:

- **Better cache locality:** Entries are stored contiguously, enabling sequential access patterns that benefit from hardware prefetching. Unlike the linked-list traversal, the processor can issue all cache-miss loads in parallel since element addresses are computable without reading previous elements (section 2.5).
- **Direct field access:** By storing raw pointers to the `Reference` object's fields at discovery time, the processing phase can write directly to those fields without re-loading the object header or navigating the object layout avoiding one level of indirection.
- **Independent of the discovered field:** The array is a separate structure, so the `discovered` field is not overloaded. To prevent re-discovery, the field is set to a self-loop sentinel.
- **Allocation cost:** Unlike the intrusive list, the array requires heap allocation for its backing storage. Growth involves copying the existing entries to a new, larger buffer.

The array approach is particularly well-suited to references that will *not* be enqueued, because these references never need to appear on the pending list. Their entire lifecycle from discovery through clearing can be handled within the GC's own data structures, without involving the `ReferenceHandler` thread at all.

2.7 ZGC Reference Processing Details

This section provides further detail on aspects of ZGC's reference processing that are directly relevant to the optimizations discussed in subsequent chapters.

2.7.1 Per-Worker Discovered Lists

In ZGC, each GC worker thread maintains its own discovered list head (via the `ZPerWorker` template). During concurrent marking, when a worker encounters a reference that should be discovered, it prepends it to its own list. Because each worker has an independent list head, **no atomic compare-and-swap (CAS) is required for discovery** a simple pointer write suffices.

This stands in contrast to the shared reference processor used by G1 and other collectors, where multiple GC threads may concurrently attempt to discover references. In the shared processor, an atomic CAS on the `discovered` field is used to resolve races: the CAS succeeds only if the field is still `null` (i.e., the reference has not already been discovered by another thread).

While correct, this introduces contention and latency, especially under high discovery rates.

2.7.2 Generational Reference Handling

Generational ZGC introduces a subtlety in reference processing that has implications for correctness. When a `Reference` object resides in the old generation but its referent is in the young generation, the referent is treated as **strongly reachable** during old-generation collection. This conservative treatment ensures that the referent is not prematurely cleared; it will be properly evaluated when the young generation is collected.

Conversely, only references in the old generation are discovered during an old-generation collection cycle. References in the young generation are skipped entirely and will be handled during a young-generation collection.

This generational boundary has practical consequences for the array-based optimization: when processing array entries, the GC must verify that the referent is in the old generation before clearing it. If the referent has been promoted to the young generation, it must be kept alive, and the entry is dropped without clearing. If young-generation marking is active, the referent may additionally need to be explicitly marked through the young-generation marking barrier to ensure it is not lost.

2.7.3 Concurrent Processing and Barriers

Because ZGC performs reference processing concurrently with the application, the collector must use barriers when reading or writing reference fields. The load barrier ensures that any pointer loaded by the GC is in the correct “color” (i.e., its metadata bits are consistent with the current GC phase). The clean barrier, used during referent clearing, writes a properly colored null value.

These barriers are essential for correctness but add overhead to each reference processing operation. The dynamic array approach stores a snapshot of the referent pointer value at discovery time, which can reduce the number of barrier operations needed during the processing phase, as the snapshot can be compared directly rather than re-applying a load barrier.

3. Method

Three orthogonal approaches were proposed to optimize the reference processing of weak references in ZGC. These approaches were implemented in ZGC in different combinations to evaluate their individual and combined impact on performance.

3.1 Optimisation Approaches

Eliminating Reference Queue

For weak references that are not created with a reference queue, the reference processor can skip the enqueue step entirely, as there will be no reference queue to enqueue to. This optimization can significantly reduce the overhead of processing weak references that do not require reference queues.

Sequetial Code Optimisations

The reference processor can be optimized to reduce the number of instructions executed or the complexity of those instructions.

Dynamic Array for Discovered References

The reference processor can use a dynamic array to store discovered references instead of linking the references together. This can reduce the overhead of managing linked lists and improve cache locality, leading to better performance.

- 3.1.1 ZGC Modifications
- 3.1.2 G1GC Modifications
- 3.2 Optimization Strategies
 - 3.2.1 Null-Queue Skip Optimization
 - 3.2.2 Data Structure Optimizations
 - 3.2.3 Synchronization Optimizations
- 3.3 Benchmarking Methodology
 - 3.3.1 Synthetic Benchmarks
 - 3.3.2 Real-World Benchmarks
- 3.4 Experimental Setup
 - 3.4.1 Hardware Configuration
 - 3.4.2 Software Configuration
 - 3.4.3 Measurement Methodology

4. Results

4.1 Synthetic Benchmark Results

4.1.1 Baseline Performance

4.1.2 Null-Queue Optimization Impact

4.1.3 Data Structure Impact

4.2 Caffeine Benchmark Results

4.2.1 Cache Operations Performance

4.2.2 GC Pause Time Analysis

4.3 Statistical Analysis

4.4 Summary of Findings

5. Discussion

5.1 Interpretation of Results

5.1.1 Performance Improvements

5.1.2 Trade-offs

5.2 Implications for JVM Development

5.2.1 Applicability to Other Collectors

5.2.2 Integration Challenges

5.3 Limitations

5.3.1 Benchmark Coverage

5.3.2 Correctness Considerations

5.3.3 Generational Complexity

5.4 Comparison with Related Work

5.5 Summary

6. Conclusion

6.1 Summary of Contributions

6.2 Answers to Research Questions

6.3 Future Work

6.3.1 Further Optimizations

6.3.2 Broader Evaluation

6.3.3 Integration with Mainline OpenJDK

6.4 Closing Remarks

References

- [1] B. Manes, “Caffeine: A high performance caching library for Java.” <https://github.com/ben-manes/caffeine>, 2024. Accessed: 2026-02-23.
- [2] T. M. Chilimbi, B. Davidson, and J. R. Larus, “Cache-conscious structure layout,” in *Proceedings of the ACM SIGPLAN 1999 Conference on Programming Language Design and Implementation (PLDI)*, pp. 1–12, ACM, 1999.